

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Opening prompt: Circle and Rectangle both have an area, but each calculates it differently. How can one method work with both?

Learning objectives

Design a class around state, behavior, and invariants.

Implement abstract methods in concrete subclasses.

Use a common type to work with different implementations.

Explain interfaces, Comparable, and natural ordering.

2

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Students should leave able to read a small UML class specification, implement an abstract superclass and concrete subclasses, use interface references, and explain a comparison contract. The Java code uses conventional named classes and a static main entry point.

Java program structure

Element	Role
Package	Organizes related types and their names
Class / interface	Defines an implementation or a contract
Field	Stores state or a class-level value
Method	Defines an operation
main method	Conventional application entry point

An application combines your types with library types.

3

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

A program can use classes from the Java API and classes supplied by its author. In the conventional application form used here, the launcher starts a selected class's public static void main(String[] args). More than one class may declare a main method. Packages organize type names.

Encapsulation

```
class Counter {
    private int value;

    public void increment() { value++; }
    public int getValue() { return value; }
}
```

Clients use operations while the class controls its state.

4

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Private instance fields hold state for each object. Public methods define the operations clients may request. Encapsulation helps maintain rules about valid state. Private members are not directly accessible from an unrelated subclass.

Class design: a Rational value

State: a numerator and a denominator.

Invariant: denominator > 0 and the fraction is reduced.

Default value: 0/1. Display form: numerator/denominator.

Arithmetic produces a new value and preserves its operands.

An immutable design exposes no setters.

5

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

This lecture uses an immutable Rational value design: private final fields, no setters, and arithmetic operations returning new Rational values. Constructor logic must reject denominator 0, reduce fractions, and normalize the sign. Implementation also needs a policy for integer overflow.

Rational: a UML-style specification

Rational implements Comparable<Rational>
− numerator : int {final} − denominator : int {final}
+ Rational() + Rational(n : int, d : int) + getNumerator() : int + getDenominator() : int + add/subtract/multiply/divide(r : Rational) : Rational + equals(o : Object) : boolean + hashCode() : int + compareTo(r : Rational) : int + toString() : String − gcd(n : int, d : int) : int

+ public − private # protected

6

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

UML visibility: + public, − private, # protected. This is a compact class specification with name, attributes, and operations. The full operations include add, subtract, multiply, divide, getters, equals(Object), hashCode(), compareTo(Rational), toString(), and private gcd(int,int). equals returns primitive boolean and overrides Object.equals(Object). Constructors normalize the representation. The private gcd helper supports reduction.

Inheritance and specialization

Type	Shared or specialized responsibility
GeometricObject	Common color and geometry operations
Circle extends GeometricObject	Radius and circular geometry
Rectangle extends GeometricObject	Width, height, and rectangular geometry

A Circle can serve as a GeometricObject.

7

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Superclass and base class are synonyms here. Subclass and derived class refer to a more specific class. A class can extend one direct superclass. Use inheritance when the subclass can satisfy the superclass contract. Otherwise composition may express the design more clearly.

Access in a subclass

Superclass member	Direct access from a subclass
public	Yes
protected	Yes, subject to protected-access rules
package-private	Only from the same package
private	No; use an accessible superclass operation

Inheritance does not bypass private access.

8

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Private superclass fields remain part of the object’s state but are not inherited as directly accessible members. Package access depends on the package. Protected access across packages has additional rules about the qualifying reference. A superclass reference exposes only the accessible members of its declared type.

Constructors and super

```
class Base {
    private final String name;
    protected Base(String name) { this.name = name; }
}

class Child extends Base {
    Child() { super("child"); }
}
```

Superclass construction initializes the shared state.

9

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Constructors are not inherited. A subclass constructor can invoke a superclass constructor with super(arguments). The examples place this call first for broad version compatibility. Without an explicit constructor invocation, the compiler normally attempts a no-argument super() call. That fails if no accessible no-argument constructor exists.

Object and commonly overridden methods

Method	Purpose
toString()	A useful string representation
equals(Object other)	The class's chosen equality relation
hashCode()	A hash consistent with equality

finalize() is deprecated for removal.

A class defines what equality means for its values.

10

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Object API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html
Object is the root of the class hierarchy. These three methods commonly participate in class design. finalize() is deprecated for removal and should not guide resource cleanup. For closeable resources, use try-with-resources where appropriate.

Equality, identity, and hashing

Question	Operation
Are these references to the same object?	a == b
Do these objects represent equal values?	a.equals(b), according to its contract
Can equal values share hash-based lookup?	Equal values must have equal hash codes

Overriding equals normally requires overriding hashCode.

11

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Object API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Object.html
For references, == tests identity. The default Object.equals also uses identity. A value class may override equals to compare its normalized value. If equals returns true, hashCode must return the same integer. A matching hash does not establish equality. The Rational design should compare its reduced numerator and denominator.

Useful library methods: Arrays

```
import java.util.Arrays;

int[] a = {9, 2, 6};
Arrays.sort(a);
System.out.println(Arrays.toString(a)); // [2, 6, 9]
int index = Arrays.binarySearch(a, 6);  // 1
```

Library methods depend on documented preconditions.

12

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

The array must be sorted with the corresponding ordering before Arrays.binarySearch. A nonnegative result is a found index; a negative result encodes an insertion point. Arrays.toString lists one-dimensional contents. This short recap connects the previous lecture with sorting user-defined objects later in this lecture.

Abstract classes and abstract methods

Construct	Role
abstract class	A partially implemented base type
Concrete method	Shared implementation available to subclasses
Abstract method	Required behavior whose implementation is deferred

A concrete subclass supplies every remaining abstract operation.

13

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

An abstract class cannot be instantiated directly. It can contain constructors, fields, concrete methods, and abstract methods. A concrete subclass must implement all inherited abstract methods unless it already inherits a suitable implementation. An abstract subclass can leave methods unimplemented.

An abstract GeometricObject

```
abstract class GeometricObject {
    private final String color;

    protected GeometricObject(String color) {
        this.color = color;
    }
    public String getColor() { return color; }
    public abstract double getArea();
    public abstract double getPerimeter();
}
```

The abstract signatures have no method body.

14

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

This base class shares immutable color state and a concrete getter, while requiring area and perimeter implementations. Objects of this class cannot be created directly, but its constructor runs when a subclass object is created.

A concrete Circle

```
class Circle extends GeometricObject {
    private final double radius;
    Circle(double radius, String color) {
        super(color);
        if (!Double.isFinite(radius) || radius < 0)
            throw new IllegalArgumentException("radius");
        this.radius = radius;
    }
    @Override public double getArea() {
        return Math.PI * radius * radius;
    }
    @Override public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}
```

The concrete subclass implements both abstract methods.

15

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

The constructor rejects negative and nonfinite radius values. Zero is allowed. Finite values can still cause floating-point overflow for extremely large dimensions; the examples use modest inputs. @Override lets the compiler check that each method overrides a superclass method.

A concrete Rectangle

```
class Rectangle extends GeometricObject {
    private final double width, height;
    Rectangle(double width, double height, String color) {
        super(color);
        if (!Double.isFinite(width) || !Double.isFinite(height)
                || width < 0 || height < 0)
            throw new IllegalArgumentException("dimensions");
        this.width = width; this.height = height;
    }
    @Override public double getArea() { return width * height; }
    @Override public double getPerimeter() {
        return 2 * (width + height);
    }
}
```

Both subclasses share one type and provide different formulas.

16

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

As with Circle, validate dimensions at construction. Both dimensions are finite and nonnegative, but extreme products may still overflow double. Rectangle is intentionally not final so that the ComparableRectangle example can extend it.

Polymorphism through the base type

```
GeometricObject[] shapes = {
    new Circle(2, "blue"),
    new Rectangle(3, 4, "green")
};

for (GeometricObject shape : shapes) {
    System.out.println(shape.getColor());
    System.out.println(shape.getArea());
}
```

The loop calls the implementation for the actual object.

17

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

The declared type determines which operations are available at compile time. The runtime class chooses the overriding instance method. For these values, the Circle area is 4π and the Rectangle area is 12. Color uses the inherited concrete getter.

Checkpoint: abstract types

```
A  new GeometricObject("blue");

B  GeometricObject[] shapes = new GeometricObject[10];

C  GeometricObject g = new Circle(2, "blue");

D  Circle c = new Rectangle(3, 4, "green");
```

Which statements compile, and what does B contain?

18

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Answers: A does not compile because GeometricObject is abstract. B compiles and contains ten null references initially. C compiles because Circle is a GeometricObject. D does not compile because Circle does not extend Rectangle. Creating an array whose element type is abstract does not instantiate the abstract class.

Interfaces define a contract

An interface names a capability and its operations.

An implementing class fulfills the required method contracts.

Unrelated classes can implement the same interface.

An interface variable can refer to an implementing object.

19

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

An interface is a reference type that specifies a role objects can fulfill. A class can implement multiple interfaces while extending one superclass. Interface references allow code to depend on a required capability rather than a specific implementation.

Defining an interface

```
interface Measurable {
    double area();
}

// The interface is the variable’s declared type.
// The concrete class will supply the implementation.
```

A signature states what clients can request.

20

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Language Specification, interfaces.
https://docs.oracle.com/javase/specs/jls/se25/html/jls-9.html
In this declaration, area() is implicitly public abstract. An implementing method must be public. A class uses implements for interfaces. An interface can extend other interfaces. The interface itself cannot be instantiated.

Implementing an interface

```
class Plot implements Measurable {
    private final double squareMeters;
    Plot(double squareMeters) {
        if (!Double.isFinite(squareMeters) || squareMeters < 0)
            throw new IllegalArgumentException("area");
        this.squareMeters = squareMeters;
    }
    @Override public double area() { return squareMeters; }
}

Measurable plot = new Plot(50);
System.out.println(plot.area()); // 50.0
```

The reference type exposes the interface's operations.

21

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

This example defines a separate immutable field of area data for a plot. Its implementation is public, matching the interface contract. Client code can refer to the object through Measurable. The same field name and parameter name are disambiguated by this.

A generic MyList contract

```
interface MyList<E> {
    E remove(int index);
    E removeFirst();
    E removeLast();
    void add(int index, E element);
    void addLast(E element);
    void addFirst(E element);
    boolean isEmpty();
    int size();
}
```

An implementation must supply the promised behavior.

22

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

These eight operations define a small list contract. E is the element type. A suggested contract: add allows positions 0 through size inclusive; remove requires 0 through size−1. Removing first or last from an empty list should have a documented failure behavior, such as NoSuchElementException. A contract also specifies results and effects, not just signatures.

Modern interfaces can include implementations

Member	Meaning
Abstract method	An implementing class supplies the behavior
default method	An inherited instance implementation
static method	An operation called on the interface name
private method	A helper for code inside the interface
Field	A public static final field

An alternative implementation of MyList.isEmpty():

```
default boolean isEmpty() { return size() == 0; }
```

Interfaces have no constructors or per-object instance fields.

23

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Language Specification, interfaces.
https://docs.oracle.com/javase/specs/jls/se25/html/jls-9.html
Interface fields are public static final. An unmodified bodyless method is public abstract. Default and static interface methods arrived in Java 8; private helper methods arrived in Java 9. Default methods are instance methods. Static methods belong to the interface. Private helpers are internal implementation support.

Comparable and natural ordering

```
// Simplified view of the existing java.lang interface:
interface Comparable<T> {
    int compareTo(T other);
}
```

Result from a.compareTo(b)	Interpretation
< 0	a comes before b
0	a and b have the same ordering position
> 0	a comes after b

24

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Comparable API. https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Comparable.html Comparable<T> is already defined in java.lang. The code is a simplified excerpt for reading the contract; students should implement the existing interface rather than declare their own java.lang package. compareTo returns a negative integer, zero, or a positive integer. The exact nonzero value is irrelevant.

A rectangle ordered by area

```
class ComparableRectangle extends Rectangle
        implements Comparable<ComparableRectangle> {
    ComparableRectangle(double w, double h) {
        super(w, h, "green");
    }
    @Override public int compareTo(ComparableRectangle other) {
        return Double.compare(getArea(), other.getArea());
    }
}
```

Inheritance supplies geometry; the interface supplies ordering.

25

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Comparable API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Comparable.html
This class reuses Rectangle and adds Comparable<ComparableRectangle>. Double.compare compares the area values. Two rectangles with different dimensions may compare as zero if their areas match. This class retains Object.equals identity, so its natural ordering is inconsistent with equals; the next slides discuss that limitation.

Sorting comparable objects

```
ComparableRectangle[] rectangles = {
    new ComparableRectangle(3, 4),
    new ComparableRectangle(1, 1),
    new ComparableRectangle(2, 3)
};
Arrays.sort(rectangles);
for (ComparableRectangle r : rectangles)
    System.out.println(r.getArea());
```

Printed areas: 1.0, 6.0, 12.0

26

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

This example sorts by ascending area: 1, 6, 12. The client can call Arrays.sort because the element type implements Comparable. No custom sorting loop is required. Arrays.sort is a static method, while compareTo is an instance method called during comparison.

The comparison contract

The sign reverses when the operands reverse.

Ordering is transitive.

A zero result defines equivalence for the ordering.

An area-only order may disagree with object equality.

Use comparison utilities instead of subtracting numeric values.

27

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Comparable API.
https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Comparable.html
The sign must reverse when operands reverse, ordering must be transitive, and comparison with a third value must behave consistently for objects that compare as zero. Consistency with equals is strongly recommended, though not required by Comparable. In the area-only teaching example, 2×6 and 3×4 compare as zero but are not equal by Object.equals. Sorted sets and maps can treat them as the same ordering key. A production design should choose and document its equality and ordering policy.

Abstract class versus interface

Feature	Abstract class	Interface
Instance state	Yes	No instance fields
Constructors	Yes, for subclass construction	None
Implemented methods	Yes	default, static, private
Unimplemented methods	Abstract methods	Abstract methods
Class relationship	Extends one direct class	Implements multiple interfaces

A class can extend a base class and implement interfaces.

28

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Reference: Java Language Specification, interfaces.
https://docs.oracle.com/javase/specs/jls/se25/html/jls-9.html
Both can act as reference types and neither can be instantiated directly. An abstract class can define instance state and constructors; an interface has neither. Abstract classes can contain concrete and abstract methods subject to ordinary Java modifier rules. Interfaces allow abstract, default, static and private methods under interface-specific rules.

Choosing a design

Need	Possible design
Share geometry state and common code	Abstract GeometricObject
Require objects to support natural ordering	Comparable<T> interface
Require a measurable capability	Measurable interface
Use another object as an internal component	Composition

Choose the relationship that matches the domain contract.

29

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Use an abstract superclass when a genuine subtype relationship benefits from shared state or implementation. Use an interface to express a role implemented by different classes. The mechanisms can be combined. Composition fits a has-a relationship and avoids promising an inappropriate subtype relationship.

Common implementation mistakes

Instantiating an abstract class or an interface.

Leaving an abstract method unimplemented in a concrete class.

Implementing a public interface operation with weaker access.

Writing equals(Rational) instead of overriding equals(Object).

Attempting to extend two classes.

30

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Examples: instantiate an abstract class, omit an inherited abstract method in a concrete class, reduce visibility when implementing an interface, annotate an overload as if it were an override, or extend two classes. @Override catches signature mismatches. Interface inheritance does not solve conflicting default implementations automatically in every case; a class may need an explicit override.

Checkpoint: a new shape

A new Triangle extends GeometricObject.

Which methods must a concrete Triangle implement?

Can a Triangle object go into GeometricObject[]?

What would enable Arrays.sort on Triangle[]?

Justify the type relationship and each required method.

31

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Suggested answer: class Triangle extends GeometricObject must provide public getArea() and getPerimeter() implementations unless it inherits suitable concrete implementations. It must call an accessible base constructor. A Triangle object can go into GeometricObject[] through subtype assignment. To define a natural ordering among Triangle values, implement Comparable<Triangle> and compareTo(Triangle). Discuss the extra data needed for a valid triangle and its perimeter.

A complete class-design argument

State the invariant and the public operations.

Use inheritance for a valid subtype relationship.

Keep shared implementation in an appropriate base class.

Use interfaces to name the required capabilities.

Document equality and ordering, then test through the contract.

32

Source: supplied Lecture02.pptx, COMP 182, Abstract Classes and Interfaces.

Close by asking students to connect Rational, GeometricObject, and Comparable. Rational highlights value invariants. Geometry highlights shared state and runtime dispatch. Comparable highlights a reusable behavioral contract. Suggested exercise: implement a finite nonnegative Square and a comparison policy, then test through a base-type reference.